

docker

parte 2: imágenes y Dockerfile

Lo que ya sabemos (semana 1)

- Qué es Docker y en qué se diferencia de una máquina virtual.
- `docker run`, `docker ps`, `docker images`, `docker pull`, `docker stop`, `docker rm`, `docker logs`, `docker exec -it`.
- Siempre usamos imágenes que ya existían en Docker Hub (`nginx`, `ubuntu` ...).

Hoy: ¿y si quiero MI propia imagen?

- Hasta ahora solo hemos *usado* imágenes de otros.
- Hoy aprendemos a **crear** nuestras propias imágenes, a medida de nuestra aplicación.
- La herramienta para definir cómo se construye una imagen es el **Dockerfile**.

¿Qué es un Dockerfile?

Un **Dockerfile** es un fichero de texto plano con una secuencia de instrucciones que le dicen a Docker, paso a paso, cómo construir una imagen.

- Se llama exactamente `Dockerfile` (sin extensión).
- Cada instrucción genera, normalmente, una nueva **capa** en la imagen.

Estructura mínima de un Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "index.js"]
```

FROM — la imagen base

- Toda imagen se construye **a partir de otra** (salvo casos muy especiales).
- `FROM ubuntu:22.04` , `FROM node:20-alpine` , `FROM python:3.12-slim` ...
- Si no se indica tag, se usa `latest` → **mejor evitarlo**, fijar siempre una versión concreta.

¿Por qué evitar `latest`?

- `latest` es solo un tag más: puede cambiar de versión real con el tiempo.
- Un build que hoy usa Node 20 podría, meses después, usar Node 22 sin que lo hayamos pedido.
- Mejor: `node:20-alpine` , `node:20.11.0-alpine` ...

WORKDIR — el directorio de trabajo

```
WORKDIR /app
```

- Establece el directorio activo para las instrucciones siguientes.
- Si no existe, Docker lo crea.
- Mejor que `RUN cd /app` (que solo afectaría a esa instrucción).

COPY — copiar ficheros al interior de la imagen

```
COPY package.json .  
COPY . /app
```

- Copia ficheros/carpetas desde el **contexto de build** al sistema de ficheros de la imagen.
- El contexto de build es la carpeta que le pasamos a `docker build` (normalmente `.`).

ADD: la "hermana mayor" de **COPY**

- Hace todo lo que hace **COPY**, y además:
 - Puede extraer automáticamente **.tar**, **.tar.gz** ...
 - Puede descargar ficheros desde una URL.
- **Recomendación**: usa **COPY** salvo que necesites específicamente eso.

.dockerignore

- Igual que `.gitignore`, pero para el contexto de build.
- Evita copiar carpetas pesadas o innecesarias (`node_modules`, `.git`, `*.log` ...).
- Hace el `docker build` más rápido y la imagen más ligera.

```
node_modules  
.git  
*.log
```

RUN — ejecutar comandos durante el build

```
RUN npm install
```

```
RUN apt-get update && apt-get install -y curl
```

- Se ejecuta **una sola vez, durante docker build** .
- Cada **RUN** genera (normalmente) una nueva capa.

CMD — el comando por defecto al arrancar

```
CMD ["node", "index.js"]
```

- Se ejecuta **al hacer** `docker run`, no durante el build.
- Si hay varios `CMD`, solo cuenta el último.
- Se puede **sobrescribir** fácilmente pasando otro comando a `docker run`.

ENTRYPOINT — el comando fijo

```
ENTRYPOINT ["python3", "app.py"]
```

- Define el programa principal que se ejecuta siempre, pase lo que pase.
- No se sobrescribe tan fácilmente como `CMD`.
- Se puede combinar: `ENTRYPOINT` fija el programa, `CMD` aporta argumentos por defecto.

RUN vs CMD vs ENTRYPOINT

Instrucción	¿Cuándo se ejecuta?
RUN	Durante el build
CMD	Al arrancar el contenedor (sobrescribible)
ENTRYPOINT	Al arrancar el contenedor (fijo)

ENV — variables de entorno

```
ENV APP_PORT=3000  
ENV NODE_ENV=production
```

- Disponibles durante el build y en los contenedores creados a partir de la imagen.
- Se pueden sobrescribir al ejecutar: `docker run -e APP_PORT=4000 mi-app`.

EXPOSE — documentar el puerto

```
EXPOSE 3000
```

- **Solo documenta** qué puerto usa la aplicación dentro del contenedor.
- **No publica nada por sí solo:** para eso sigue haciendo falta `-p` en `docker run`.

Ejemplo completo de Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
ENV PORT=3000
EXPOSE 3000
CMD ["node", "index.js"]
```

docker build — construir la imagen

```
docker build -t mi-app:1.0 .
```

- **-t** → nombre y tag de la imagen.
- **.** → el **contexto de build** (la carpeta actual).
- **-f** → permite indicar un Dockerfile con otro nombre/ubicación.

La caché de capas

- Docker reutiliza capas ya construidas si no han cambiado ("Using cache").
- Si una capa cambia, se invalida **esa capa y todas las siguientes**.
- Por eso el **orden de las instrucciones importa muchísimo**.

Orden recomendado: lo que cambia menos, primero

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install          ← capa cara, cambia poco
COPY . .                 ← capa barata, cambia mucho
CMD ["node", "index.js"]
```

Así, si solo cambia el código, **no hace falta reinstalar dependencias.**

Buenas prácticas de imágenes

- Usar imágenes base ligeras (`alpine` , `slim`) cuando sea posible.
- Agrupar instrucciones `RUN` relacionadas para minimizar capas.
- Limpiar cachés de paquetes en la misma instrucción `RUN` que instala.
- No ejecutar como `root` si no hace falta (`USER`).
- Usar `.dockerignore` siempre.

Multi-stage builds (idea básica)

```
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Compilamos en una etapa "pesada" y copiamos solo el resultado a la imagen final, mucho más ligera.

Tags y versionado

- Un **tag** identifica una variante/versión concreta de una imagen: `nginx:1.25`, `node:20-alpine`.
- Sin tag → se usa `latest` por defecto.
- Formato de versionado habitual: **semver** (`mayor.menor.parche`, ej. `2.3.1`).

docker tag — renombrar/retaguear

```
docker tag mi-app:1.0 jose/mi-app:1.0
```

- Una misma imagen (mismo ID) puede tener **varios tags**.
- Necesario antes de subir una imagen a Docker Hub con tu usuario.

Volúmenes: persistir datos

- El sistema de ficheros de un contenedor **desaparece** cuando se elimina el contenedor.
- Un **volumen** permite persistir datos más allá del ciclo de vida del contenedor.

```
docker run -v /datos/host:/datos/contenedor mi-app
```

Bind mount vs volumen gestionado

	Bind mount (<code>-v ruta_host:ruta_cont</code>)	Volumen gestionado (<code>docker volume create</code>)
Ruta	La elige el usuario, en el host	La administra Docker
Uso típico	Desarrollo, compartir código	Datos de producción (BBDD...)

Bind mount: tu proyecto real dentro del contenedor

El **bind mount** no es solo "un tipo de volumen": es la pieza clave para desarrollar cómodamente con Docker.

- Montas la carpeta de tu proyecto (en el host) dentro del contenedor.
- Los cambios que haces en tu editor, **fuera** del contenedor, se ven al instante **dentro**.
- No hace falta reconstruir la imagen cada vez que cambias una línea de código.

```
docker run -it -v $(pwd):/app -w /app mi-app:1.0 bash
```

El flujo típico de desarrollo con bind mount

1. La imagen (`mi-app:1.0`) solo lleva el **entorno**: lenguaje, dependencias del sistema, herramientas.
2. El **código real** del proyecto vive en tu máquina (y en su propio repositorio git).
3. Con `-v` , montas ese código dentro del contenedor en tiempo de ejecución.
4. Editas en tu máquina → pruebas dentro del contenedor → repites, sin volver a hacer `docker build` .

Solo reconstruyes la imagen si cambia el **entorno** (una dependencia nueva, una versión de lenguaje...), no cada vez que cambias código.

Redes básicas y `-p`

```
docker run -p 8080:80 nginx
```

- `-p host:contenedor` → publica el puerto del contenedor en el host.
- Por defecto, un contenedor se conecta a la red **bridge** de Docker.
- Dos contenedores en la **misma red personalizada** pueden comunicarse por **nombre**.

Docker Hub: publicar nuestra imagen

```
docker login  
docker tag mi-app:1.0 jose/mi-app:1.0  
docker push jose/mi-app:1.0
```

- `docker login` → autenticarse.
- El nombre de la imagen debe incluir el usuario/repositorio de destino.
- `docker pull jose/mi-app:1.0` la descarga desde cualquier sitio.

Cuidado con lo que metes en una imagen

- Todo lo que copias o escribes en capas queda dentro de la imagen, **aunque luego lo borres en una capa posterior.**
- Nunca metas contraseñas o claves reales en `ENV` ni en el código copiado si la imagen es pública.

Organización de repositorios Git con Docker

Una disyuntiva habitual

Cuando trabajas con Docker y bind mount, suelen aparecer **dos carpetas con Dockerfile /entorno y código real**, no una:

- Una carpeta "workspace" con el `Dockerfile` y todo lo necesario para levantar el entorno de un framework (p. ej. `laravel-workspace`, `node-workspace`).
- La carpeta del proyecto real montada dentro vía bind mount (p. ej. `laravel-workspace/mi-proyecto`).

¿Mantienes **un único workspace reutilizable** para todos tus proyectos de ese framework, o creas **un repositorio nuevo por proyecto** que incluya también su propio entorno?

Dos repositorios git, dos propósitos distintos

- El repositorio del **workspace** versiona el entorno (`Dockerfile` , scripts de arranque...). Muchas veces **no hace falta subirlo** a ningún sitio especial: es solo tu caja de herramientas local.
- El repositorio del **proyecto** (dentro del workspace, montado por bind mount) es el que de verdad importa versionar y compartir: ahí vive el código de la aplicación.
- Son `.git` **independientes**: uno en la carpeta del workspace, otro en la carpeta del proyecto. Git no "ve" más allá de su propia carpeta, así que no hay conflicto entre ambos.

¿Cuál elegir?

- **Workspace reutilizable:** cómodo si trabajas a menudo con el mismo framework/stack; un solo `Dockerfile` te sirve para todos tus proyectos de ese tipo.
- **Repositorio por proyecto** (con su propio `Dockerfile` dentro): mejor si cada proyecto necesita un entorno algo distinto, o si quieres que cualquiera pueda clonar el proyecto y tener el entorno listo sin pasos extra.
- No hay una única respuesta correcta: depende de cuánto varíe el entorno entre tus proyectos.

Comandos que veremos hoy

- `docker build -t nombre:tag .`
- `docker tag`, `docker push`, `docker pull`, `docker login`
- `docker run -p host:contenedor -v host:contenedor -e VAR=valor imagen`
- `docker history`

A practicar

Dockerfile propio, build, volúmenes, puertos y Docker Hub